API Testing Interview Questions

1. What is an API? How do APIs work?

Suggested Approach: You can start with a concise definition and then elaborate on the communication flow in APIs. Maybe throw in an example as well.

Sample Answer: An API (Application Programming Interface) is a defined specification of the rules and protocols that allow different applications to interact with each other. It specifies the procedures and data formats for applications to communicate information.

Basically, the workflow of any API is that the client sends a request to it, and the API works as an intermediary. The request is received by the API, it processes the request, and returns a response to the client. This response usually includes the information that was requested or confirmation of the action that was performed. APIs create an abstraction layer in such a way that the actual implementation details are hidden from the outside world and only the relevant functionalities are exposed.

2. Explain the components of an API request.

Suggested Approach: Try to break down a typical API request into its essential parts. Maybe throw in an example as well.

Sample Answer: An API request generally has a few important parts, such as:

- Endpoint (URL/URI): An endpoint is the point of entry for the API, the address/URL/URI where the API is available. This resource is the one that the client wants to interact with.
- HTTP Method (Verb): Action that should be performed on the resource (GET, POST, PUT, DELETE, PATCH, etc.)
- Headers: Key/value pairs that provide metadata about the request and response. Common headers include:
- Content-Type: The type of data being sent in the request.
- Accept: Indicates the desired media type of the response from the server.
- Authorization: It carries authentication details.
- User-Agent: Name of the client making the request.
- Body (Payload): The sent data if any will accompany the request over here. Most probably used with the POST, PUT, and PATCH methods. Holds the actual data typically as a JSON object or XML.
- Query Parameters: Optional key-value pairs added to the URL after the '? for filtering, sorting or paginating results.
- Path Parameters: A value embedded into the URL Path that refers to a specific resource.

3. What is API Testing? Why is it important?

Suggested Approach: Define API testing, then highlight its benefits and why it's a vital part of the testing cycle.

Sample Answer: API testing is a type of software testing that focuses on validating the functionality, reliability, performance, and security of APIs. Unlike UI testing, which interacts with the graphical user interface, API testing directly accesses the application's business logic layer.

It is important because it helps with:

- Early defect detection
- Faster feedback loop
- Decoupled testing
- Improved test coverage
- Enhanced reliability

4. What are the different types of API tests?

Suggested Approach: List and briefly explain various categories of API testing to show that you have a comprehensive understanding.

Sample Answer: Common types include:

- Functional Testing: Confirms that the API is correctly performing its intended functions by sending requests and validating responses against expectations. These consist of positive, negative, and edge-case testing.
- Performance Testing: Assesses the responsiveness, stability, and capacity of the API in different load scenarios (load testing, stress testing, spike testing, endurance testing) by measuring response time, throughput, and error rate.
- Security Testing: Discover vulnerabilities in the API, including authentication bypasses, injection (SQL, XSS) flaws, broken access control, and data encryption.
- Reliability Testing: Verifies the capability of the API to connect and conduct its function under certain scenarios.
- Load Testing: Testing the API under the anticipated user loads.
- Stress Testing: Making the API work harder than it typically would, driving it to or even past its breaking point.
- Fuzz Testing: Sending random or unexpected inputs to API in order to identify vulnerabilities or crashes
- Validation Testing: Ensures the API returns the expected results in the required format.
- UI Testing (in context of API): UI testing is not exactly API testing, however it is a combination of making sure the UI connects to or talks with and shows data from and API.
- Interoperability Testing: Ensures that the API is able to communicate with other APIs or systems as intended.
- Contract Testing: Tests that the API conforms to an agreement between the consumer and the producer.

5. What is the difference between REST and SOAP APIs?

Suggested Approach: For this, the simplest option is to create a list of major differences by architecture style, message format, protocol, and flexibility.

Sample Answer: REST stands for Representational State Transfer and is stateless and lightweight in utilizing HTTP methods. Best for Web Services & JSON Data. SOAP (Simple Object Access Protocol) – a protocol that provides a uniform way of passing data between services and uses XML for its message format. REST is used more for web and mobile applications because it is simple and flexible, whereas SOAP is preferred in enterprise environments for strict contracts and advanced security features.

6. What are the common HTTP methods used in API testing?

Suggested Approach: Over here, list the primary HTTP methods and provide a concise explanation for each.

Sample Answer:

- GET: Retrieves data from the server. It's a read-only operation and should be idempotent (multiple identical requests have the same effect as a single one).
- POST: Submits data to the server to create a new resource. It's not idempotent (multiple identical POST requests can create multiple resources).
- PUT: Updates an existing resource or creates a new one if it doesn't exist. It's idempotent (repeated PUT requests will have the same effect). It typically replaces the entire resource.
- PATCH: Partially updates an existing resource. It's not necessarily idempotent and only modifies the specified fields.
- DELETE: Removes a specified resource from the server. It's idempotent.

7. What are HTTP Status Codes, and why are they important in API testing?

Suggested Approach: To answer this question, explain what status codes are, provide examples for different categories (2xx, 4xx, 5xx), and emphasize their role in validating API responses.

Sample Answer: HTTP status codes are 3-digit numbers which are returned from the server in response to an API request indicating whether the request was accomplished or failed. These are important in API testing, as they can quickly tell you if the request was successful, whether there was a client-side error, or a server-side error.

Some key types of status codes are:

- 2xx (Success)
- 200 OK: The request was successful.
- 201 Created: A new resource was successfully created (typically for POST requests).
- 204 No Content: The request was successful, but there's no content to return (e.g., a successful DELETE).
- 4xx (Client Error)

- 400 Bad Request: The server cannot process the request due to malformed syntax or invalid data.
- 401 Unauthorized: Authentication is required or has failed.
- 403 Forbidden: The client does not have permission to access the resource.
- 404 Not Found: The requested resource does not exist on the server.
- 5xx (Server Error)
- 500 Internal Server Error: A generic error occurred on the server.
- 502 Bad Gateway: The server, while acting as a gateway or proxy, received an invalid response from an upstream server.
- 503 Service Unavailable: The server is currently unable to handle the request due to temporary overload or maintenance.

8. What do you think needs to be verified in API testing?

Suggested Approach: List the key aspects to be validated when automating an API, covering functional and/or non-functional areas.

Sample Answer: While doing API testing, a number of things need to be validated to confirm the quality of the API.

- Functionality:
- Data Accuracy: Does the API return the correct data as per the request?
- Business Logic: Is the API appropriately enforcing the underlying business rules and calculations?
- Input Validation: What does the API do in the case of valid, invalid, absent or edge-case inputs?
- Response Structure: Are the correct error messages and status codes returned for invalid requests or failure?
- Error Handling: Are appropriate error messages and status codes returned for invalid requests or failures?
- Reliability: The API should continue doing its job across varying conditions.
- Performance:
- Response Time: How fast does the API respond to requests?
- Throughput: What would be the number of requests that the API can process in a given period of time?
- Scalability: How does the API scale when the load increases?
- Security:
- Authentication & Authorization: Can only authenticated users/systems access the resources? Is sensitive data protected?
- Vulnerability Testing: Assess against common vulnerabilities like SQL injection, XSS, CSRF, etc.
- Usability/Documentation: Is the API documentation clear and easy to understand for consumers?
- Compliance: Is the API compliant with industry standards or regulations?

9. What are some common tools used for API testing?

Suggested Approach: List widely used tools, categorize them if possible (e.g., manual, automation, performance), and briefly mention their strengths.

Sample Answer: Several tools are widely used for API testing, each with its strengths:

- Postman: A very popular and user-friendly GUI-based tool for manual and automated API testing. It allows sending requests, inspecting responses, organizing tests into collections, and creating test scripts.
- SoapUI: An open-source tool primarily used for testing SOAP web services, but also supports REST. It's excellent for functional, performance, and security testing.
- REST Assured: A Java-based library that simplifies the testing of RESTful APIs. It's highly popular for building robust and readable API automation frameworks in Java.
- JMeter: Primarily a performance testing tool, but it can also be used for functional API testing by sending HTTP/HTTPS requests and analyzing responses.
- Swagger/OpenAPI: While not strictly testing tools, they provide excellent API documentation that can be used to understand and generate basic tests. Tools like Swagger UI allow sending requests directly from the documentation.
- curl: A command-line tool for transferring data with URLs, commonly used for quick manual API calls and scripting.

10. How do you handle authentication and authorization in API testing?

Suggested Approach: Explain different authentication methods and how you would apply them in a testing context. Also, emphasize the importance of testing various access levels.

Sample Answer: Handling authentication and authorization is critical in API testing to ensure only legitimate users/systems can access resources and perform allowed actions. My approach involves:

- Understanding Authentication Mechanisms:
- API Keys: Often passed in headers or query parameters. I'd include these directly in the request.
- Basic Auth: Username and password sent Base64 encoded in the Authorization header.
- OAuth (1.0/2.0): Involves obtaining access tokens, usually through a separate authentication flow. I'd automate this token generation and pass the bearer token in the Authorization header.
- JWT (JSON Web Tokens): Similar to OAuth, involves obtaining and sending a token. I'd parse the token to ensure its structure and validity.
- Test Scenarios:
- Positive Testing: Verify successful access with valid credentials for different user roles (e.g., admin, regular user).
- Negative Testing:
- Sending requests without any authentication.
- Using invalid/expired tokens or incorrect credentials.

- Attempting to access resources with insufficient permissions (e.g., a regular user trying to delete an admin resource).
- Token Refresh/Expiration: Testing how the API handles expired tokens and the refresh token mechanism.
- Tools: Most API testing tools like Postman and Rest Assured have built-in support for various authentication types. This simplifies the process of configuring and sending authenticated requests.

11. What is API mocking, and why is it used in API testing?

Suggested Approach: Over here, you should define API mocking, explain its purpose, and provide scenarios where it's beneficial.

Sample Answer: API mocking is the practice of simulating the behavior of a real API or external service by creating a controlled, fake version of it. Instead of sending requests to the actual backend, test requests are directed to the mock API, which returns predefined responses. They are useful for testing in early development stages.

12. Explain the need for API documentation.

Suggested Approach: Highlight the various stakeholders who benefit from documentation and explain its importance for development, testing, and consumption.

Sample Answer: API documentation is a critical component for the success of any API. It serves as a comprehensive guide that outlines how to effectively use and interact with an API. It helps

- Provides the necessary information (endpoints, methods, parameters, authentication, response formats) for developers to integrate with the API efficiently and correctly. Without it, integration becomes a frustrating process of trial and error.
- Testers rely on documentation to understand the expected behavior, valid inputs, error conditions, and response structures, which are all crucial for designing effective test cases.
- Simplifies debugging and maintenance by providing a clear understanding of the API's design and dependencies.
- The documentation (especially if following standards like OpenAPI/Swagger) acts as a living contract between the API provider and its consumers.

13. How do you approach testing an API that lacks documentation?

Suggested Approach: Emphasize a systematic, exploratory approach and the importance of collaboration and documenting findings.

Sample Answer: API documentation provides the necessary information on endpoints, request formats, expected responses, and status codes. Testing an undocumented API can be challenging, but I'd approach it systematically through:

Exploratory Testing: Start by exploring common endpoints and HTTP methods. I'd use tools like Postman or Insomnia to send GET requests to common paths, and try basic POST/PUT/DELETE operations.

- Observe Responses: Carefully analyze the response bodies, status codes, and headers to understand the API's structure, data formats (JSON, XML), and error handling.
- Reverse Engineer: Based on observations, try to deduce the API's business logic, expected inputs, and resource relationships.
- Collaborate with Developers: This is crucial. I'd actively communicate with developers to clarify ambiguities, understand intended functionality, and gain insights into the API's design and dependencies.
- Incremental Documentation: As I discover endpoints and their behaviors, I'd start creating my own internal documentation or test case descriptions. This helps in building a knowledge base and ensuring consistency.
- Focus on Core Functionality: Prioritize testing the most critical endpoints and functionalities first.
- Parameter Fuzzing: Once a basic understanding is formed, I'd experiment with various parameter combinations, valid and invalid data, to uncover unexpected behaviors.
- Regression Testing: As more is learned and the API evolves, establish a strong set of regression tests to ensure stability.

Additional Resources:

- Exploratory Testing vs. Scripted Testing: Tell Them Apart and Use Both
- Exploratory Testing: How Do You Approach it as a QA Engineer?
- What is a Test Charter?
- What is Regression Testing?
- The Difference Between Regression Testing and Retesting
- Smoke Testing vs Regression Testing – Key Differences You Need to Know
- Automated Regression Testing

14. What is API contract testing? Why is it important?

Suggested Approach: Define contract testing, explain its role in microservices, and highlight its benefits.

Sample Answer: API contract testing is a type of testing that ensures that the interactions between a consumer (client) and a provider (API) adhere to a predefined agreement or 'contract' regarding the API's behavior. This contract typically specifies the request format, expected response format, data types, and status codes. It's particularly important in microservices architectures where multiple services integrate with each other.

Additional Resources: API Contract Testing: A Step-by-Step Guide to Automation

15. How do you handle dynamic values in API testing (e.g., timestamps, unique IDs)?

Suggested Approach: Explain techniques like variable extraction, parameterization, and handling specific dynamic data types.

Sample Answer: Dynamic values, such as timestamps, session IDs, authentication tokens, or newly generated resource IDs, are common in API responses and often need to be used in subsequent requests. I handle them using the following techniques:

- Variable Extraction/Chaining:
- In tools like Postman, I'd use pm.environment.set() or pm.collectionVariables.set() in the 'Tests' script section to extract a value from the response body or headers and store it in a variable. This variable can then be used in subsequent requests.
- In automation frameworks (e.g., Rest Assured), I'd parse the JSON or XML response using libraries like JsonPath or XPath and store the extracted value in a programmatic variable.
- Parameterization: For dynamic values that are part of the request payload or URL, I'd parameterize them. This involves using variables or data sources (e.g., CSV, Excel) to feed different dynamic values into test cases.
- Generating Dynamic Data:
- Timestamps: For timestamps, I'd generate them at runtime using programming language functions (e.g., Date.now() in JavaScript, System.currentTimeMillis() in Java) or specific timestamp functions in the testing tool.
- Random Data: For random strings or numbers, I'd use random data generation functions or libraries provided by the testing framework.
- Regular Expressions: For complex patterns or values embedded within a string, regular expressions can be used to extract the required dynamic parts from the response.

Additional Resources: Dynamic Data in Test Automation: Guide to Best Practices

16. What are the challenges in API testing?

Suggested Approach: Discuss common hurdles faced during API testing, demonstrating your awareness of practical difficulties.

Sample Answer: API testing comes with its own set of challenges:

- Handling authentication and authorization correctly.
- Dealing with different data formats.
- Ensuring compatibility with different versions of the API.
- Testing performance under high load.

17. How do you decide what test cases to write for an API?

Suggested Approach: Describe a systematic approach to test case design, focusing on coverage and different testing types.

Sample Answer: My approach to designing API test cases is systematic and aims for comprehensive coverage:

- Understand API Documentation (Swagger/OpenAPI): This is the starting point. I'd thoroughly review the documentation to understand each endpoint, its purpose, supported HTTP methods, request parameters (required/optional, data types, constraints), and expected response structures (including status codes).
- Identify Functional Scenarios:
- Positive Scenarios: Valid requests with correct data, ensuring successful operations (e.g., creating a user, retrieving data).
- Negative Scenarios: Invalid inputs (wrong data types, missing required fields, out-of-range values), unauthorized access attempts, non-existent resources, malformed requests, to check error handling.
- Edge Cases/Boundary Conditions: Testing with minimum/maximum values, empty strings, nulls, special characters.
- Authentication & Authorization: Test access for different user roles, valid/invalid tokens, and cases where authentication is missing.
- Data Validation: Verify that the API correctly validates input data against its schema and returns appropriate errors for invalid data.
- Response Verification:
- Status Codes: Verify correct HTTP status codes (2xx, 4xx, 5xx).
- Response Body: Validate the data returned against expected values and its structure.
- Headers: Check for relevant headers like Content-Type and Cache-Control.
- Performance & Load: Identify critical endpoints that might experience high traffic and design performance test cases (though usually separate).
- Security: Consider common vulnerabilities and design test cases to probe for them (e.g., SQL injection, XSS if input reflects in output).
- Chained Requests/Workflows: For interdependent APIs, design test cases that simulate realistic user flows, passing data from one API response to the subsequent request.
- Error States: Explicitly test scenarios that should lead to errors, such as network timeouts, database connection issues (if possible to simulate).
- Data Clean-up: For tests that create or modify data, consider test cases for data clean-up to maintain test environment integrity.

Additional Resources: How to Write Test Cases? (+ Detailed Examples)

18. How do you integrate API tests into a CI/CD pipeline?

Suggested Approach: Explain the benefits of CI/CD integration and describe the steps involved, including tools.

Sample Answer: Integrating API tests into a CI/CD (Continuous Integration/Continuous Delivery) pipeline is essential for achieving continuous quality and faster feedback. My approach involves:

- Automate Tests: Ensure all API tests are automated using a framework (like Rest Assured, Newman for Postman collections, or a custom script) that can be executed programmatically.
- Version Control: Store all test scripts and test data in a version control system (e.g., Git) alongside the application code.
- CI Tool Integration: Configure the CI server (e.g., Jenkins, GitLab CI, Azure DevOps, CircleCI) to trigger API tests automatically.
- Commit Triggers: The pipeline should be configured to run API tests on every code commit or pull request merge to catch regressions early.
- Scheduled Builds: Optionally, schedule nightly or periodic runs for more extensive test suites.
- Environment Setup: Ensure the CI environment has access to the necessary test data, credentials, and the deployed API instance. This might involve containerization (Docker) for consistent environments.
- Test Execution: The CI pipeline will execute the automated API tests.
- Reporting: Integrate a reporting mechanism (e.g., TestNG/JUnit reports, Allure Reports, Newman's HTML reporter) to generate clear and concise test results.
- Notifications: Configure the CI tool to send notifications (e.g., email, Slack) to the team on test failures, allowing for quick action.
- Gates/Quality Gates: Set up quality gates in the pipeline that prevent further deployment if critical API tests fail, ensuring that only quality code moves forward.

Additional Resources: What Is CI/CD?

19. How do you handle data-driven testing for APIs?

Suggested Approach: Define data-driven testing, explain its benefits for APIs, and detail the methods and tools used.

Sample Answer: Data-driven testing in APIs involves executing the same API test case multiple times with different sets of input data. This approach is highly effective for testing various scenarios without writing redundant code and for ensuring the API behaves correctly across a wide range of data inputs.

My approach typically involves:

- Identify Data Sources:
- CSV Files: Simple and widely used for small to medium datasets. Each row represents a new set of test data.
- Excel Files: Similar to CSV but can handle more complex data structures and multiple sheets.
- Databases: For larger, more complex, or frequently changing data, querying a database directly for test data is efficient.
- JSON/XML Files: Can be used to store complex nested data structures for request payloads.
- Parameterization: The key is to parameterize the API requests. Instead of hardcoding values in the request URL, headers, or body, placeholders are used.

- In Postman, this involves using variables like {{variable_name}} in requests and then providing data files (CSV/JSON) during collection runner execution.
- In automation frameworks (e.g., Rest Assured with TestNG/JUnit, Python with requests): Data is read from the external source, and then programmatic loops or data providers iterate through the data, injecting it into the API call.
- Test Case Design:
- Design a single test case that can handle the parameterized inputs.
- Ensure assertions are robust enough to validate expected outputs based on the specific input data.

Additional Resources: Data-driven Testing: How to Bring Your QA Process to the Next Level?

20. How do you test API error handling?

Suggested Approach: Explain the importance of error handling and detail systematic steps for testing various error scenarios.

Sample Answer: Testing API error handling is crucial to ensure that the API responds gracefully and informatively when things go wrong, whether due to client mistakes or server issues. My approach involves:

- Categorize error responses based on expected error scenarios
- Design test cases for each of these scenarios
- Automate these test cases

Additional Resources: Effective Error Handling Strategies in Automated Tests

21. What is JSON and how do you test it in API?

Suggested Approach: Define JSON, explain how to validate it, and highlight its advantages for API communication.

Sample Answer: JSON (JavaScript Object Notation) is a lightweight, human-readable, and machine-parsable data interchange format. When testing APIs that use JSON, I focus on several aspects:

- Verify that the JSON response (or request body) conforms to a predefined JSON schema.
- Validate the JSON data for correctness, completeness, and data types.
- Check for the presence of mandatory fields and the absence of fields that should not be returned under specific circumstances.
- Ensure that error responses are also in valid JSON format and contain expected error codes and messages.
- Assess the time taken to parse and process JSON responses, especially for large payloads.

22. How do you test response time in API testing?

Suggested Approach: Explain why response time testing is important, outline the process, and mention tools and metrics.

Sample Answer: Testing response time in API testing is a critical aspect of performance testing, ensuring that the API delivers data within acceptable limits for a smooth user experience and efficient system integration.

My process for testing response time involves:

- Defining performance benchmarks by collaborating with stakeholders.
- Pinpoint the APIs that are most frequently called, handle large data volumes, or are crucial to core business functionality, as these will have the most significant impact on overall system performance.
- Choose the right tools to do the job.
- Design performance test cases to check the load and stress that the API can handle.
- Once the results are back, check that they satisfy the metrics.
- If response times are consistently above benchmarks, investigate potential bottlenecks in the API code, database queries, network latency, or server resources.
- Present the findings clearly, comparing actual performance against established benchmarks, highlighting any areas of concern.

23. What is the role of headers in API testing?

Suggested Approach: Define headers, explain their purpose, and list common headers and their significance in API testing scenarios.

Sample Answer: HTTP headers are key-value pairs that provide metadata about the request or response in an API communication. They are not part of the actual data payload but convey important information about the transaction. In API testing, headers are important because they help with:

- Authentication and authorization
- Content negotiation
- Caching control
- Request information
- Tracing and debugging

24. How do you perform load testing on APIs?

Suggested Approach: Define load testing for APIs, outline the steps involved, discuss tools, and mention key metrics.

Sample Answer: Load testing on APIs is a type of performance testing that evaluates the API's behavior under an expected or peak level of concurrent user load. The primary goal is to ensure the API can handle the anticipated traffic volume without significant performance degradation or errors.

My approach to performing load testing on APIs typically involves:

- Determine which API endpoints are most frequently accessed or crucial for the application's core functionality.
- Estimate the number of concurrent users, requests per second (RPS), or transactions per hour based on historical data, user projections, or marketing campaigns.
- Next is to select a suitable load testing tool.
- Create test scripts that accurately simulate API calls, including correct HTTP methods, endpoints, headers (especially authentication tokens), and request bodies.
- Configure the load testing parameters like number of virtual users, ramp-up period, duration, and loop count.
- Run the load test from one or more load generators (depending on the scale) to simulate distributed user traffic.
- Monitor and collect data.

25. How do you prioritize API test cases for regression testing?

Suggested Approach: Explain the importance of prioritization for regression and detail a systematic approach using criteria like criticality, defect history, and usage patterns.

Sample Answer: Prioritizing API test cases for regression testing is crucial to ensure that changes or new features don't inadvertently break existing functionality, especially when time or resources are limited. A strategic approach helps focus efforts on the most impactful areas. Here's how I prioritize:

- Criticality or impact on business flow
- High-value features
- Frequency of use/ high traffic endpoints
- Recent changes or modifications
- Know areas of defect density/ bug history
- APIs that are dependencies for many other services or applications
- APIs with complex business logic, calculations, or data transformations